

计算机系统结构实验报告Lab05:

MIPS Single-Cycle CPU

uu-matter543

2024-2025-2

摘要

在 Lab05 中，我完成了 MIPS 32 单周期处理器的各部件连接，实现了 9 条基本指令：lw, sw, add, sub, and, or, beq, slt, j 并进行了验证；在 9 条指令基础上扩展了 7 条指令：addi, andi, ori, sll, srl, jal, jr，实现了 16 条指令，并对扩展 7 条指令进行了验证；利用已实现的 16 条指令实现了乘法器并进行了仿真，最后将二进制流文件烧录至开发板并实现了拨码乘法器，收获颇丰。

目录

1 实验目的	3
1.1 简单的类 MIPS 单周期处理器的工作原理	3
1.2 简单的类 MIPS 单周期处理器的设计与实现	3
1.3 仿真测试与上板验证	3
2 基本 9 条指令的实现与验证	3
2.1 新模块的添加和已有模块的修改	3
2.1.1 Zero Extend 模块的添加	3
2.1.2 Instruction Memory 模块的添加	3
2.1.3 PC Control Unit 的添加	4
2.1.4 Data Memory 的修改	5
2.2 Top 模块的设计和已有模块的连接	5
2.2.1 Top 模块的对外端口映像	5
2.2.2 Control Unit 的连接	6
2.2.3 ALU Control Unit 的连接	7
2.2.4 Registers 的连接	7

2.2.5	位扩展模块的连接	8
2.2.6	ALU 的连接	8
2.2.7	Data Memory 的连接	9
2.2.8	PC 及其控制	10
2.2.9	Instruction Memory 的连接	10
2.3	基本 9 条指令的验证	10
2.3.1	数据内存设置	10
2.3.2	指令内存设置	11
2.3.3	仿真激励文件编写与仿真实现	13
3	扩展 7 条指令的实现与验证	13
3.1	扩展 7 条指令的支持	13
3.1.1	addi, andi, ori 功能添加	13
3.1.2	sll, srl 功能添加	17
3.1.3	jal 功能添加	18
3.1.4	jr 功能添加	19
3.2	扩展 7 条指令的验证	20
3.2.1	内存设置	20
3.2.2	仿真激励文件编写与仿真实现	21
4	MIPS 32 单周期处理器的应用：简单乘法器	21
4.1	汇编指令分析	21
4.2	内存设置	23
4.2.1	数据内存设置	23
4.2.2	指令内存设置	23
4.3	仿真实现与结果展示	24
5	上板验证	24
5.1	设计文件的修改	24
5.1.1	显示模块的添加及 CPU 的相应修改	24
5.1.2	内存的配置	27
5.1.3	最后调整	29
5.2	结果展示	29
6	总结与反思	31

1 实验目的

1.1 简单的类 MIPS 单周期处理器的工作原理

需要重点关注的内容包括：

1. 指令执行时的数据通路；
2. 对应的控制线路；
3. 各功能部件间的互联定义；
4. 逻辑选择关系。

1.2 简单的类 MIPS 单周期处理器的设计与实现

1. 支持基本 9 条指令 (lw, sw, beq, add, sub, and, or, slt, j) 的 CPU 的实现与调试
2. 扩展至支持 16 条指令 (增加 addi, andi, ori, sll, srl, jal, jr) 的 CPU 的设计与实现

1.3 仿真测试与上板验证

以乘法汇编代码为例，进行仿真测试与上板验证

2 基本 9 条指令的实现与验证

2.1 新模块的添加和已有模块的修改

2.1.1 Zero Extend 模块的添加

```
module zero_extend(  
    input  [15:0] data_16,  
    output [31:0] data_32  
);  
    assign data_32 = {{16{1'b0}}, data_16};  
endmodule
```

在指令执行过程中，除了需要符号位扩展，在逻辑与、逻辑或运算中也需要零扩展，即将 16 位立即数的高 16 位以 0 填充。实现 Zero Extend 模块的思路类似于实现 Sign Extend 模块的思路，只需要修改补充位为 0 即可。

2.1.2 Instruction Memory 模块的添加

```
module instruction_memory(  
    input reset,  
    input [31:0] addr,  
    output [31:0] instr  
);  
    reg [31:0] mem_cont [63:0];  
    assign instr = reset ? mem_cont[0] : mem_cont[addr>>2];  
endmodule
```

MIPS 32 架构属于 Harvard Architecture, 指令与数据分别储存在两个不同的内存模块中, 因此要仿照 Data Memory 模块的设计, 添加 Instruction Memory 模块。定义端口 `reset` 用于重置指令为第 1 条指令, 定义端口 `addr` 为输入地址, 定义端口 `instr` 为指令输出端口; 在内部定义一组 64 个 32 位寄存器, 以模拟指令内存的存储部件。输入输出关系定义为, 重置时 `instr` 输出第 1 条指令, 非重置时输出 `addr` 指定地址上的指令。由于实际内存都是字节寻址的, 而本实验中模拟的 Memory 均以 32 位寄存器为一个单元, 因此要将输入地址右移 2 位以忽略 2 个低位, 这一点在后面的 Data Memory 模块修改中也会考虑到。

2.1.3 PC Control Unit 的添加

```
module pc_control_unit(  
    input [31:0] old_pc,  
    input jump,  
    input branch,  
    input zero,  
    input reset,  
    input [25:0] j_targ,  
    input [31:0] beq_offset,  
    output [31:0] new_pc  
);  
  
    wire [31:0] pc_add_4;  
    assign pc_add_4 = old_pc + 4;  
    assign new_pc = reset ? 0 :  
        (jump ? {pc_add_4[31:28], j_targ<<2} :  
         (branch && zero ? (beq_offset<<2) + pc_add_4 :  
          pc_add_4));  
endmodule
```


为了方便管理 PC 的迭代情况，我将 PC 的控制单独作为一个模块加以实现。

定义输入端口 `old_pc` 接收前一条指令的 PC，定义输入信号 `reset` 作为复位信号，定义输入信号 `jump`, `branch`, `zero` 分别作为 `j` 指令信号、`beq` 指令信号、`beq` 跳转条件信号，定义输入端口 `j_targ` 作为 `j` 指令的目标地址，定义输入端口 `beq_offset` 作为 `beq` 指令的偏移量，定义输出端口 `new_pc` 作为本条指令更新后 PC 的值。在内部定义了 `pc_add_4` 数据通路，因为 MIPS 32 架构中 PC 的更新以 `PC+4` 的值为基础。

输入输出关系定义为，重置时将 PC 设为 0，以同步 Instruction Memory 的重置输出第 1 条指令；非重置时，如果是 `j` 指令，则 `new_pc` 是以 `PC+4` 的最高 4 位为最高 4 位、以 26 位 `j_targ` 作为中间 26 位、以 2 位 0 作为最低 2 位组合而成的值；不是 `j` 指令时，如果是 `beq` 指令，且跳转条件满足，则 `new_pc` 为 `beq_offset` 左移 2 位后加上 `PC+4` 的值；如果不是 `beq` 指令，或跳转条件不满足，则简单输出 `PC+4` 即可。

2.1.4 Data Memory 的修改

因为仿真过程中仿真模块会从文件中读取 `mem_cont` 值，不需要初始化，因此去掉了设计文件中的初始化内容；另外，与 Instruction Memory 设计思想类似，由于实际内存都是字节寻址，因此将输入地址均右移 2 位，从而与汇编器翻译出的机器码保持一致。基于以上 2 点，将 Data Memory 模块的输入输出关系修改如下：

```
// Before Modification
reg[31:0] mem_cont[63:0];
integer i;
initial for(i = 0; i < 64; i = i + 1) mem_cont[i] <= 0;
assign rd_data = mem_rd && !mem_wr ? mem_cont[addr] : 0;
always @ (negedge clock) if (mem_wr) mem_cont[addr] = wr_data;

// After Modification
reg [31:0] mem_cont [63:0];
assign rd_data = mem_rd && !mem_wr ? mem_cont[addr>>2] : 0;
always @ (negedge clock) if (mem_wr) mem_cont[addr>>2] = wr_data;
```

2.2 Top 模块的设计和已有模块的连接

2.2.1 Top 模块的对外端口映像

```
module Top(
    input Reset,
    input Clk
);
```

```
wire [31:0] Instruction;
```

定义 Top 模块，以实现内部模块的封装，便于实现实际功能或对其进行扩展。

输入端口比较简单，只需要输入端口 Clk 作为系统时钟同步各模块的工作，以及输入端口 Reset 作为复位信号的输入；同时定义了内部数据通路 Instruction 作为指令，因为由多模块所用，非常重要，而紧跟输入模块定义。

2.2.2 Control Unit 的连接

```
wire RegDst;
wire Jump;
wire Branch;
wire MemRead;
wire MemWrite;
wire [1:0] ALU_Op;
wire MemToReg;
wire ALUSrc;
wire RegWriteEn;
control_unit Ctr(
    .op_code(Instruction[31:26]),
    .reg_dst(RegDst),
    .alu_src(ALUSrc),
    .mem_to_reg(MemToReg),
    .reg_write(RegWriteEn),
    .mem_read(MemRead),
    .mem_write(MemWrite),
    .branch(Branch),
    .alu_op(ALU_Op),
    .jump(Jump)
);
```

已有端口即输入 op_code，为 Instruction 的 [31:26] 位，连接即可。新定义数据通路：

- RegDst 选择寄存器写操作的目标寄存器下标为 Instruction 的 [20:16] 或 [15:11] 位；
- Jump 为无条件跳转指令信号；
- Branch 为有条件跳转指令信号；
- MemRead 为数据内存读使能信号；
- MemWrite 为数据内存写使能信号；
- 2 位 ALU_Op 为输入到 ALU Control Unit 的 2 位控制信号；
- MemToReg 选择寄存器写操作的数据为 ALU 的计算结果或从数据内存中读取的数据；

- ALUSrc 选择 ALU 第二个操作数的数据源为寄存器数据端口;
- RegWriteEn 为寄存器写使能信号。

将内部数据通路依次与 Control Unit 的各个输出端口连接。

2.2.3 ALU Control Unit 的连接

```
wire [3:0] ALUCtrOut;  
alu_control_unit ALU_Ctr(  
    .funct(Instruction[5:0]),  
    .alu_op(ALU_Op),  
    .alu_ctr_out(ALUCtrOut)  
);
```

已有数据通路连接为:

- 2 位 alu_op 来自 Control Unit;
- 6 位 funct 来自 Instruction 的 [5:0] 位。

新定义 4 位数据通路 ALUCtrOut 为对 ALU 执行操作的控制信号, 与 ALU Control Unit 的对应输出端口连接即可。

2.2.4 Registers 的连接

```
wire [31:0] RegData1;  
wire [31:0] RegData2;  
wire [31:0] WriteData;  
wire [4:0] WriteReg;  
assign WriteReg = RegDst ?  
    Instruction[15:11] :  
    Instruction[20:16];  
  
registers Reg(  
    .clock(Clk),  
    .reset(Reset),  
    .rd_reg_1(Instruction[25:21]),  
    .rd_reg_2(Instruction[20:16]),  
    .wr_reg(WriteReg),  
    .rd_data_1(RegData1),  
    .rd_data_2(RegData2),  
    .wr_data(WriteData),  
    .wr_en(RegWriteEn)
```

```
);
```

已有数据通路连接:

- clock 来自系统时钟;
- reset 来自复位信号;
- 2 个读寄存器下标分别来自 Instruction 的 [25:21] 和 [20:16] 位;
- 写使能信号来自 Control Unit。

新定义数据通路:

- RegData1, RegData2 分别为根据读寄存器下标读取的数据;
- WriteData 为寄存器写的数据源;
- WriteReg 是 RegDst 信号对指令 [20:16] 或 [15:11] 的选择结果, 也是写寄存器下标。

将内部数据通路与寄存器组的端口连接, 并保持读数据可在任意时刻, 而写数据只能在时钟下降沿的准则不变, 防止产生同步问题。

2.2.5 位扩展模块的连接

```
wire [31:0] AfterSignExt;  
sign_extend SignExt(  
    .data_16(Instruction[15:0]),  
    .data_32(AfterSignExt)  
);  
  
wire [31:0] AfterZeroExt;  
zero_extend ZeroExt(  
    .data_16(Instruction[15:0]),  
    .data_32(AfterZeroExt)  
);
```

已有数据通路即 Instruction 的 [15:0] 位即 16 位立即数, 作为两个位扩展模块的输入。新定义数据通路: AfterSignExt 为符号扩展后的 32 位数据, AfterZeroExt 为零扩展后的 32 位数据。

2.2.6 ALU 的连接

```
wire [31:0] ALUOperand2;  
wire [31:0] ALUResult;  
wire Zero;  
assign ALUOperand2 = ALUSrc ? AfterSignExt :  
    RegData2;
```

```
alu ALU(  
    .reg_1(RegData1),  
    .reg_2(ALUOperand2),  
    .alu_ctr(ALUCtrOut),  
    .alu_result(ALUResult),  
    .zero(Zero)  
);
```

已有数据通路连接为：

- 第 1 个操作数来自寄存器组的第 1 个读数据端口；
- 控制信号来自 ALU Control Unit 的输出。

新定义数据通路：

- ALUOperand2 为第 2 个操作数的来源，由 ALUSrc 选择为位扩展结果（因为 9 条基本指令的实现无需零扩展，为便于初期调试，只选用了位扩展结果），或寄存器组的第 2 个读端口；
- ALUResult 为 ALU 的运算结果
- Zero 为 ALU 运算结果的零标记位，可用于条件跳转的条件判断。

2.2.7 Data Memory 的连接

```
wire [31:0] RdDataMem;  
data_memory DataMem(  
    .clock(Clk),  
    .addr(ALUResult),  
    .mem_wr(MemWrite),  
    .wr_data(RegData2),  
    .mem_rd(MemRead),  
    .rd_data(RdDataMem)  
);
```

已有数据通路：

- clock 来自系统时钟；
- addr 来自 ALU 的运算结果；
- mem_wr 为来自 Control Unit 的写使能信号；
- wr_data 来自 Registers 的第 2 个读数据端口，是 Data Memory 写操作的数据源；
- mem_rd 为来自 Control Unit 的读使能信号。

新定义数据通路 RdDataMem 为 Data Memory 读操作读取的数据，与端口连接即可。

2.2.8 PC 及其控制

```
reg [31:0] PC;
wire [31:0] NewPC;
pc_control_unit PCContr(
    .old_pc(PC),
    .jump(Jump),
    .branch(Branch),
    .reset(Reset),
    .zero(Zero),
    .j_targ(Instruction[25:0]),
    .beq_offset(AfterSignExt),
    .new_pc(NewPC)
);
always @ (posedge Clk) PC <= NewPC;
```

定义内部寄存器 PC 作为当前的 PC 值，连接 PC Control Unit 的各控制信号。新定义数据通路 NewPC 作为 PC Control Unit 向 PC 传输数据的通路，并设置为在时钟上升沿更新 PC，以贴合单周期 CPU 每条指令执行 1 个周期的功能。

2.2.9 Instruction Memory 的连接

```
instruction_memory InstrMem(
    .reset(Reset),
    .addr(PC),
    .instr(Instruction)
);
assign WriteData = MemToReg ? RdDataMem : ALUResult;
```

Instruction Memory 按照前述内容连接即可。最后，补充由 MemToReg 信号选择：从数据内存中读取的数据或 ALU 的运算结果，并将选中内容作为寄存器写操作的数据源 WriteData 的数据选择机制。

至此，Top 模块和各子模块的连接均已完成。

2.3 基本 9 条指令的验证

2.3.1 数据内存设置

为便于调试和验证，将数据内存第 0 ~ 7 个 Word 分别设为 0 ~ 7，后续各 Word 均为 0，如图所示（17 ~ 64 行均为 0，已省略）；为提高编写效率，数据内存使用了 16 进制。

```
E: > Desktop > Project5-9Instructions > Project5.sim > sim_1 > behav > xsim > ≡ lab05dat_tb
1    00000000
2    00000001
3    00000002
4    00000003
5    00000004
6    00000005
7    00000006
8    00000007
9    00000000
10   00000000
11   00000000
12   00000000
13   00000000
14   00000000
15   00000000
16   00000000
```

图 1: 基本 9 条指令验证的数据内存

2.3.2 指令内存设置

为展现细节，指令内存使用了 2 进制。图中指令的含义分别为（未提及的为空指令）：

- 01: $\text{lw } \text{Mem}[04] \rightarrow \text{Reg}[1];$
- 02: $\text{lw } \text{Mem}[08] \rightarrow \text{Reg}[2];$
- 03: $\text{add } \text{Reg}[3] \leftarrow \text{Reg}[2] + \text{Reg}[1];$
- 04: $\text{sub } \text{Reg}[4] \leftarrow \text{Reg}[2] - \text{Reg}[1];$
- 05: $\text{sw } \text{Reg}[3] \rightarrow \text{Mem}[32];$
- 06: $\text{sw } \text{Reg}[4] \rightarrow \text{Mem}[36];$
- 07: $\text{lw } \text{Mem}[12] \rightarrow \text{Reg}[1];$
- 08: $\text{lw } \text{Mem}[20] \rightarrow \text{Reg}[2];$
- 09: $\text{and } \text{Reg}[5] \leftarrow \text{Reg}[2] \text{ AND } \text{Reg}[1];$
- 10: $\text{or } \text{Reg}[6] \leftarrow \text{Reg}[2] \text{ OR } \text{Reg}[1];$
- 11: $\text{sw } \text{Reg}[5] \rightarrow \text{Mem}[40];$
- 12: $\text{sw } \text{Reg}[6] \rightarrow \text{Mem}[44];$
- 13: $\text{beq } (\text{Reg}[4] == \text{Reg}[5]) \Rightarrow \text{Jump } 2(\text{to } 16);$
- 16: $\text{beq } (\text{Reg}[3] == \text{Reg}[6]) \Rightarrow \text{Jump } 16(\text{to } 33);$
- 17: $\text{slt } (\text{Reg}[3] == \text{Reg}[6]) \Rightarrow (\text{Reg}[7] \leftarrow \text{Reg}[1]);$
- 18: j Jump to 18;

```
E: > Desktop > Project5-9Instructions > Project5.sim > sim_1 > behav > xsim > ≡ lab05instr_tb
1  10001100000000010000000000000100
2  10001100000000010000000000000100
3  00000000001000100001100000100000
4  0000000001000001001000000100010
5  10101100000000110000000000100000
6  10101100000001000000000000100100
7  10001100000000010000000000001100
8  100011000000000100000000000010100
9  00000000001000100010100000100100
10 00000000001000100011000000100101
11 10101100000001010000000000101000
12 10101100000001100000000000101100
13 00010000100001010000000000000010
14 00000000000000000000000000000000
15 00000000000000000000000000000000
16 00010000011001100000000000010000
17 00000000001000100011100000101010
18 0000100000000000000000000010001
19 00000000000000000000000000000000
20 00000000000000000000000000000000
21 00000000000000000000000000000000
22 00000000000000000000000000000000
23 00000000000000000000000000000000
24 00000000000000000000000000000000
25 00000000000000000000000000000000
26 00000000000000000000000000000000
27 00000000000000000000000000000000
28 00000000000000000000000000000000
29 00000000000000000000000000000000
30 00000000000000000000000000000000
31 00000000000000000000000000000000
32 00000000000000000000000000000000
33 10001100000111110000000000000100
```

图 2: 基本 9 条指令验证的指令内存

- 33: $lw\ Reg[31] \rightarrow Mem[04];$

2.3.3 仿真激励文件编写与仿真实现

```
Top CPU(  
    .Clk(sys_clk),  
    .Reset(reset)  
);  
always #(100) sys_clk = !sys_clk;  
initial begin  
    sys_clk = 0;  
    reset = 0;  
    $readmemh ("lab05dat_tb", CPU.DataMem.mem_cont);  
    $readmemb ("lab05instr_tb", CPU.InstrMem.mem_cont);  
    #100; reset = 1;  
    #100; reset = 0;  
end
```

编写仿真激励文件 `instr_9_tb.v`，主要激励如上。其中 `$readmemh` 和 `$readmemb` 语句分别表示从指定文件中以 16 进制和 2 进制读取内容，存放至已定义模块的指定位置。

用到的激励有：时钟初始化及频率设置，`reset` 开启并关闭以复位各模块，从文件中读取数据内存和指令内存的内容。之后便可以从波形图上看到各指令的执行情况。

第 1 ~ 6 条指令 (`lw`, `add`, `sub`, `sw`) 执行情况如图3 (100ns ~ 1300ns) 所示，第 7 ~ 12 条指令 (`lw`, `and`, `or`, `sw`) 执行情况如图4 (1300ns ~ 2500ns) 所示。

剩余指令 (`beq`, `slt`, `j`) 执行情况如图5 (2500ns ~ 3300ns) 所示。其中第 13 条指令 (`PC = 30H`) 中跳转条件满足，直接跳转到第 16 条指令 (`PC = 3CH`)；第 16 条指令 (`PC = 3CH`) 中跳转条件不满足，于是顺序执行；第 17 条指令 (`PC = 40H`) 中满足 `slt` 条件，7 号寄存器被设为 1；第 18 条指令 (`PC = 44H`) 为无条件跳转到自己，于是进入死循环，`PC = 44H` 不变，防止 `PC` 无限增加导致问题。

以上仿真表明 9 条基本指令的功能已正确实现。

3 扩展 7 条指令的实现与验证

3.1 扩展 7 条指令的支持

3.1.1 `addi`, `andi`, `ori` 功能添加

这 3 条指令都是 I 型指令，相比于 R 型指令，每类 I 型指令都有自己的 `OpCode`，因此要从 Control Unit 开始加以调整。

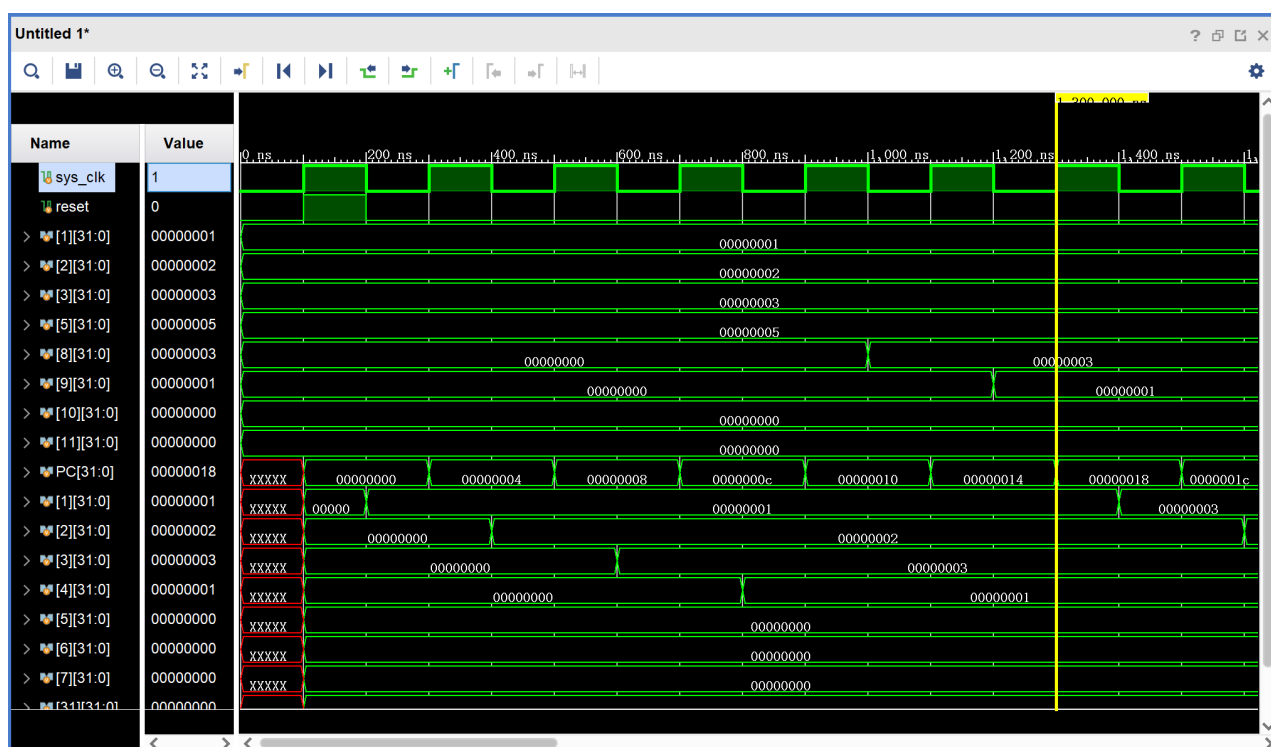


图 3: 指令 lw, add, sub, sw 的验证

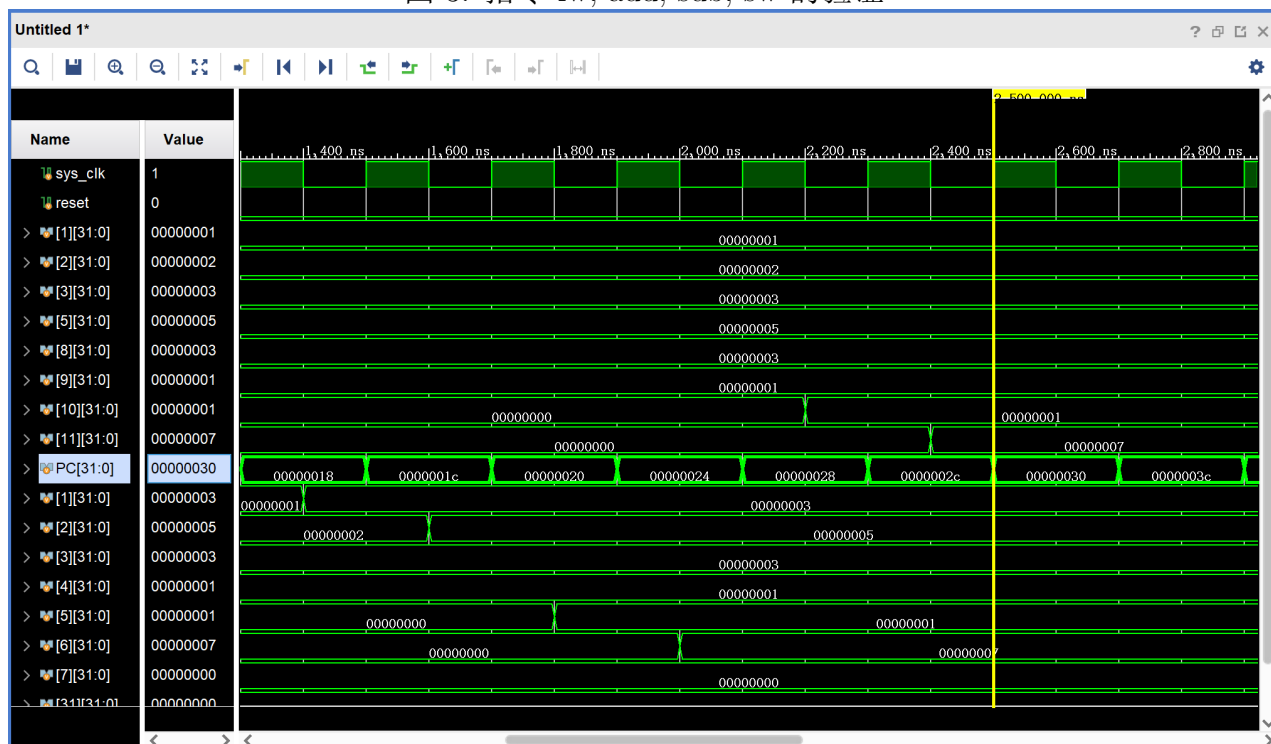


图 4: 指令 lw, and, or, sw 的验证

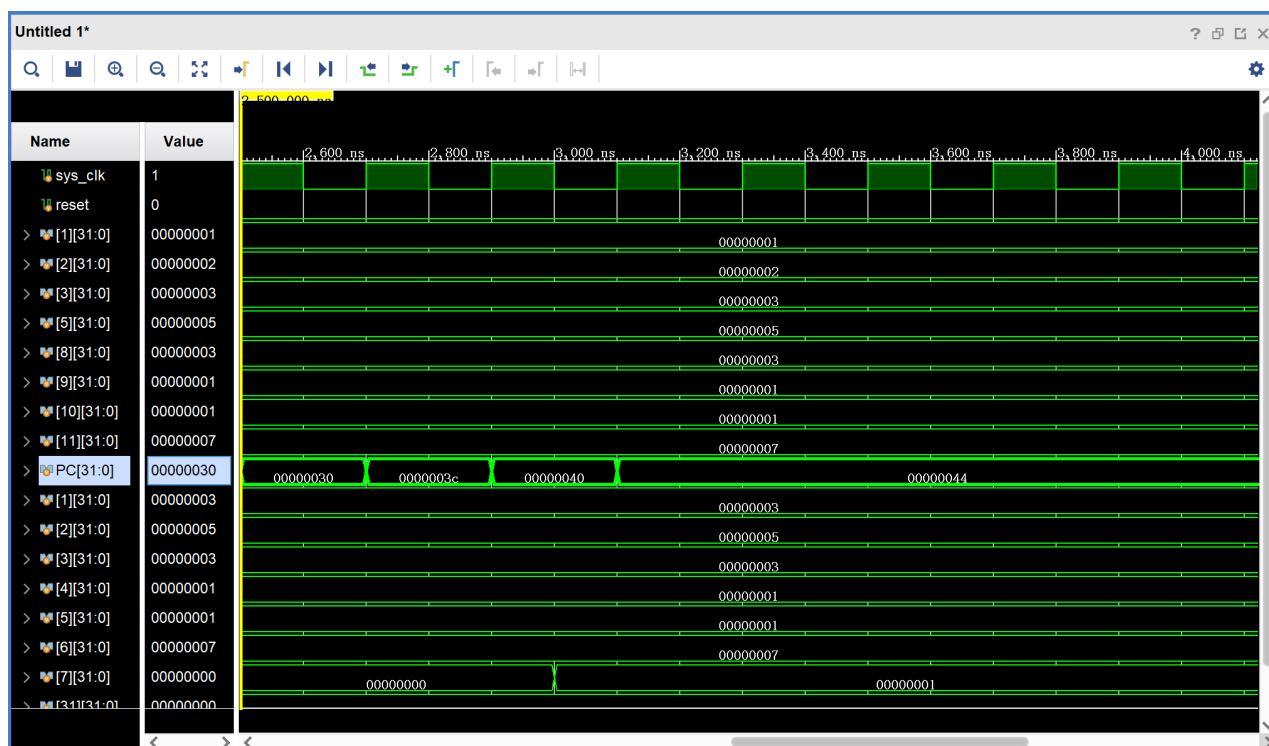


图 5: 指令 beq, slt, j 的验证

Control Unit 调整如下:

```

6'b001000: begin // addi
    reg_dst = 0;
    alu_src = 1;
    mem_to_reg = 0;
    reg_write = 1;
    mem_read = 0;
    mem_write = 0;
    branch = 0;
    alu_op = 3'b000;
    jump = 0;
    sign_ext = 1;
    jump_load = 0;
end
6'b001100: begin //andi
    reg_dst = 0;
    alu_src = 1;
    mem_to_reg = 0;
    reg_write = 1;

```

```

        mem_read = 0;
        mem_write = 0;
        branch = 0;
        alu_op = 3'b010;
        jump = 0;
        sign_ext = 0;
        jump_load = 0;
end
6'b001101: begin // ori
        reg_dst = 0;
        alu_src = 1;
        mem_to_reg = 0;
        reg_write = 1;
        mem_read = 0;
        mem_write = 0;
        branch = 0;
        alu_op = 3'b011;
        jump = 0;
        sign_ext = 0;
        jump_load = 0;
end

```

因为 `addi` 执行过程中对立即数进行符号扩展，`andi` 和 `ori` 执行过程中对立即数进行零扩展，因此要利用到实现 9 条指令时添加的 Zero Extend 模块，并添加输出信号 `sign_ext` 以对 2 种扩展的结果加以选择，这一点也会在 ALU 的修改中提到。

而 `addi`，`andi`，`ori` 都要根据 opcode 直接决定 ALU 中进行的运算，加上 9 条指令中的 `beq`，所以要根据 opcode 选择：加、减、逻辑与、逻辑或、R 型指令总共 5 种情况，所以若要由 ALU Control Unit 直接控制 ALU 中的运算，`alu_op` 仅有 2 位是不够的，因此将其扩展为 3 位，这一点也会在 ALU Control Unit 的修改中提到。

至于 `jump_load` 信号，将在 `jal` 指令的实现中加以说明。

ALU Control Unit 的调整如下：

```

// I-type
9'b000xxxxxx: alu_ctr_out <= 4'b0010; //add
9'b001xxxxxx: alu_ctr_out <= 4'b0110; //sub
9'b010xxxxxx: alu_ctr_out <= 4'b0000; //and
9'b011xxxxxx: alu_ctr_out <= 4'b0001; //or

```

由于 `alu_op` 由 2 位扩展为 3 位，因此 ALU Control Unit 的输入信号也从 8 位扩展为 9 位。代码中的 9 位信号由 Control Unit 的 3 位输出 `alu_op` 和 Instruction 的 [5:0] 位

funct 顺序排列而成。

由于 I 型指令的最低 6 位是 16 位立即数的最低 6 位，可能是任意情况，且与 ALU 操作的无关，因此后 6 位要写为无关位，仅由 Control Unit 的 3 位输出 alu_op 决定 ALU 操作。又因为 Control Unit 仅由 Instruction 的 [31:25] 位决定，即实现了由 opcode 直接决定 ALU 中的操作。未涉及到的 alu_op 为 1xx 的情况，对应 R 型指令，将基本 9 条指令中 alu_op 和 funct 的各种情况修改为 9 位即可。

ALU 的调整如下：(在 Top.v 中)

```
assign ALUOperand2 = ALUSrc ? (SignExtend ? AfterSignExt :  
                                AfterZeroExt):  
                                RegData2;
```

由于加、减、逻辑与、逻辑或操作均已在 9 条指令中实现，因此 ALU 内部暂时无需调整，仅在 Top 模块中把 ALU 的第 2 个操作数修改为：由 ALUSrc 信号选择为经位扩展后的立即数，或从寄存器读端口得到的寄存器值；若选中了位扩展后的立即数，则由 SignExtend 信号选择为经符号扩展后的立即数，或经零扩展后的立即数。

3.1.2 sll, srl 功能添加

这 2 条指令都是 R 型指令，无需修改 Control Unit，直接从 ALU Control Unit 开始修改即可。

ALU Control Unit 的调整如下：

```
// R-type  
9'b1xx100000: alu_ctr_out <= 4'b0010; //add  
9'b1xx100010: alu_ctr_out <= 4'b0110; //sub  
9'b1xx100100: alu_ctr_out <= 4'b0000; //and  
9'b1xx100101: alu_ctr_out <= 4'b0001; //or  
9'b1xx101010: alu_ctr_out <= 4'b0111; //slt  
9'b1xx000000: alu_ctr_out <= 4'b1110; //sll  
9'b1xx000010: alu_ctr_out <= 4'b1111; //srl
```

在 9 条指令中 add, sub, and, or, slt 的基础上，添加根据 sll, srl 的 funct 位产生 ALU 控制字信号的逻辑，由于指令表中并未对 alu_ctr_out 与 ALU 中运算的对应关系加以说明，这里可以自定义，注意不要与 9 条指令中 ALU 操作的控制字重复即可，我定义的操作与 ALU 控制字的关系如代码所示。

另外，Lab03 中实现的 ALU Control Unit 将 funct 最高 2 位输入视为无关位，仅根据最低 4 位确定 ALU 操作，而 sll, srl 的 funct 的最低 4 位与已实现的 ALU 操作有重复，因此每个 casex 子句必须将 funct 的 6 位明确定义，以避免冲突和重复。

ALU 的调整如下：

```
4'b1110: alu_result = reg_2 <<< shamt;
4'b1111: alu_result = reg_2 >>> shamt;
```

因为 2 条位移指令的操作的位数由 shamt 决定，因此需在输入中添加 5 位端口 shamt，并与 Top 模块中 Instruction 的 [10:6] 位相连接，这里不再赘述。最后对应 ALU 控制字，添加对操作数的操作即可。另外，注意 sll 和 srl 指令为逻辑位移操作，对应 Verilog 语言的运算符分别为：左移 <<<，右移 >>>。

3.1.3 jal 功能添加

这条指令是 J 型指令，与 I 型指令类似，jal 指令也有不同于其它指令的 OpCode，也需要 Control Unit 为其专门添加控制代码块；而由于 J 型指令无需 ALU 参与，因此无需修改 ALU Control Unit 和 ALU。

Control Unit 调整如下：

```
6'b000011: begin // jal
    reg_dst = 0;
    alu_src = 0;
    mem_to_reg = 0;
    reg_write = 1;
    mem_read = 0;
    mem_write = 0;
    branch = 0;
    alu_op = 3'b000;
    jump = 1;
    sign_ext = 0;
    jump_load = 1;
end
```

由于 jal 指令涉及 PC 向寄存器写入，寄存器写操作又有了新的数据源，要添加信号对其加以选择。因此向 Control Unit 再添加 jal 的输出信号 jump_load，当且仅当指令为 jal 时 jump_load 为 1，其余输出信号与 j 指令相同。

Registers 调整如下：（在 Top.v 中）

```
assign WriteReg = JumpLoad ? 5'b11111 :
    (RegDst ? Instruction[15:11] :
    Instruction[20:16] );
assign WriteData = JumpLoad ? PC :
    (MemToReg ? RdDataMem :
    ALUResult );
```

Registers 内部也无需调整，只需要将写数据端口的数据源和目标寄存器的选择关系加以修改，添加 JumpLoad 信号选择将 PC 写入 31 号寄存器即可。

3.1.4 jr 功能添加

jr 是 R 型指令，无需修改 Control Unit，直接从 ALU Control Unit 开始修改即可。
ALU Control Unit 和 ALU 的调整如下：

```
// ALU Control Unit
9'b1xx001000: alu_ctr_out <= 4'b0100; //jr
// ALU
4'b0100: alu_result = 32'h00000000;
```

由于 R 型指令中寄存器写使能信号是开启的，而写寄存器的下标为 0，为避免错误，直接通过 2 级控制，将 ALU 运算结果强制设为 0。因为 j 指令执行无需 branch 的选择，所以 ALU 的 Zero 标志位为 0 并不会产生新的问题。

PC 控制逻辑的调整：

```
// Top
wire JumpReg;
pc_control_unit PCContr(
    .old_pc(PC),
    .jr_reg(RegData1),
    .jump(Jump),
    .jump_reg(JumpReg),
    .branch(Branch),
    .reset(Reset),
    .zero(Zero),
    .j_targ(Instruction[25:0]),
    .beq_offset(AfterSignExt),
    .new_pc(NewPC)
);
always @ (Reset) if (Reset) PC <= 0;
always @ (posedge Clk) PC <= NewPC;
assign JumpReg = (Instruction[5:0] == 001000) ? 1 : 0;
// PC Control Unit
assign new_pc = jump_reg ? jr_reg :
    (jump ? {pc_add_4[31:28], j_targ<<2} :
    (branch && zero ? (beq_offset<<2) + pc_add_4 :
    pc_add_4));
```

由于支持 jr 指令后, PC 更新关系更为复杂, 因此将 PC 重置操作外置于 Top 模块, PC Control Unit 只负责在非重置时进行 PC 的迭代。

添加 32 位输入端口 jr_reg, 表示 jr 指令执行时读取到的寄存器值, 并与寄存器组的第 1 个读数据端口连接。对于 PC 的更新, 添加 JumpReg 信号, 当且仅当指令的 funct 与 jr 的 001000 相等时 JumpReg 为 1; 同时在原有数据选择基础上, 添加 jump_reg 信号对 PC 更新源的选择, 在 jump_reg 为 1 时选择以寄存器值更新 PC。

至此, 扩展 7 条指令的功能支持均已完成。

3.2 扩展 7 条指令的验证

3.2.1 内存设置

因为不涉及数据内存操作, 所以将数据内存设置为 64 个 32 位 0, 不再赘述。指令内存设置如下:

```
E: > Desktop > Project5-16Instructions > Project5.sim > sim_1 > behav > xsim > lab05instr_tb
1  00100000000000001000000000000011
2  00100000000000001000000000000101
3  00110000001000110000000000000101
4  00110100010001000000000000000011
5  000000000000000010010100001000000
6  0000000000000000100011000000100010
7  0010000000000011100000000000100000
8  00000000111000000000000000001000
9  00001100000000000000000000001001
10 00001000000000000000000000001001
```

图 6: 扩展 7 条指令验证的指令内存

图中指令的含义分别为:

- 01: $\text{addi } \text{Reg}[1] \leftarrow \text{Reg}[0] + 3;$
- 02: $\text{addi } \text{Reg}[2] \leftarrow \text{Reg}[0] + 5;$
- 03: $\text{andi } \text{Reg}[3] \leftarrow \text{Reg}[1] \text{ AND } 0005\text{H};$
- 04: $\text{ori } \text{Reg}[4] \leftarrow \text{Reg}[2] \text{ OR } 0003\text{H};$
- 05: $\text{sll } \text{Reg}[5] \leftarrow \text{Reg}[1] \ll 1;$
- 06: $\text{srl } \text{Reg}[6] \leftarrow \text{Reg}[2] \gg 1;$
- 07: $\text{addi } \text{Reg}[7] \leftarrow \text{Reg}[0] + 8;$
- 08: jr Jump to $\text{Reg}[7]$ (to 9);
- 09: jal $\text{Reg}[31] \leftarrow \text{PC}$, Jump to 10;
- 10: j Jump to 10;

3.2.2 仿真激励文件编写与仿真实现

仿真激励文件 `instr_16_tb.v` 的编写与验证 9 条基本指令所用仿真文件 `instr_9_tb.v` 完全相同，这里不再赘述，仅展示指令执行情况。第 1 ~ 7 条指令 (`addi`, `andi`, `ori`, `sll`, `srl`) 的执行情况如图7 (100ns ~ 1500ns) 所示，第 8 ~ 10 条指令 (`jr`, `jal`, `j`) 的执行情况如图8 (1500ns ~ 2100ns) 所示。

以上仿真表明 7 条扩展指令的功能已正确实现。

4 MIPS 32 单周期处理器的应用：简单乘法器

4.1 汇编指令分析

模拟整型正数乘法的汇编代码如下：

```
lw $1, 0($3)
lw $2, 4($3)
srl $4, $2, 1
sll $5, $4, 1
beq $5, $2, 1
add $8, $8, $1
srl $2, $2, 1
sll $1, $1, 1
beq $2, $3, 1
j 2
sw $8, 8($3)
```

执行乘法的步骤为：

1. `lw` 指令将数据内存第 1 个字，第 2 个字分别载入 `$1`, `$2` 寄存器表示 2 个乘数；在本段汇编代码中，`$3` 寄存器的值恒为 0，作为哨兵元素；
2. `srl`, `sll` 指令，将 `$2` 寄存器的值右移 1 位保存到 `$4`，将 `$4` 寄存器的值左移 1 位保存到 `$5`；若 `$2` $\neq$ `$5`，则 `$2` 寄存器值的最低位为 1，否则 `$2` 寄存器的最低位为 0；
3. 根据移位相加的乘法计算方式，若 `$2` 寄存器的最低位为 1，结果寄存器 `$8` 要加上 `$1` 寄存器的当前值，否则什么也不做；并在执行计算后将 `$2` 寄存器的值右移 1 位，表示该位已计算；将 `$1` 寄存器的值左移 1 位，是后续计算的可能用到的参数；
4. 当 `$2` 寄存器值为 0 时，表示计算已完成；将 `$8` 的值写回内存第 3 个字，就可以在内存第 3 个字查看到计算结果。

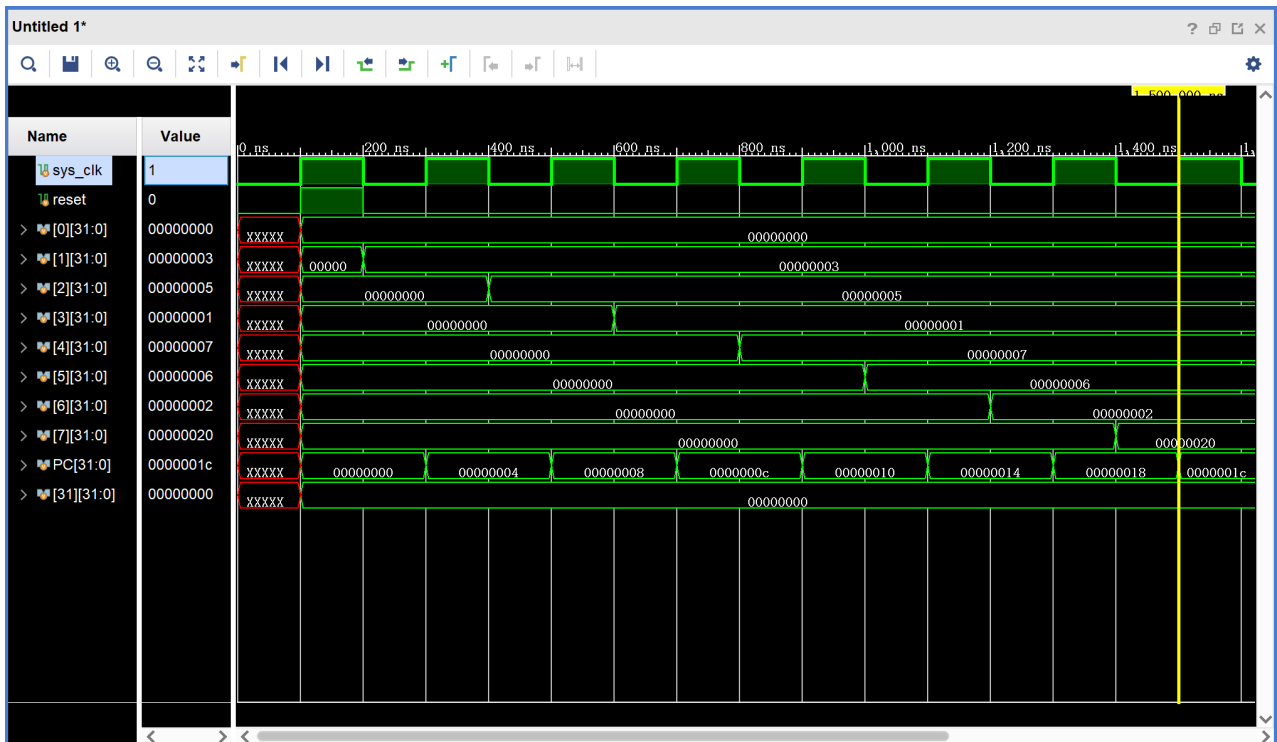


图 7: 指令 addi, andi, ori, sll, srl 的验证

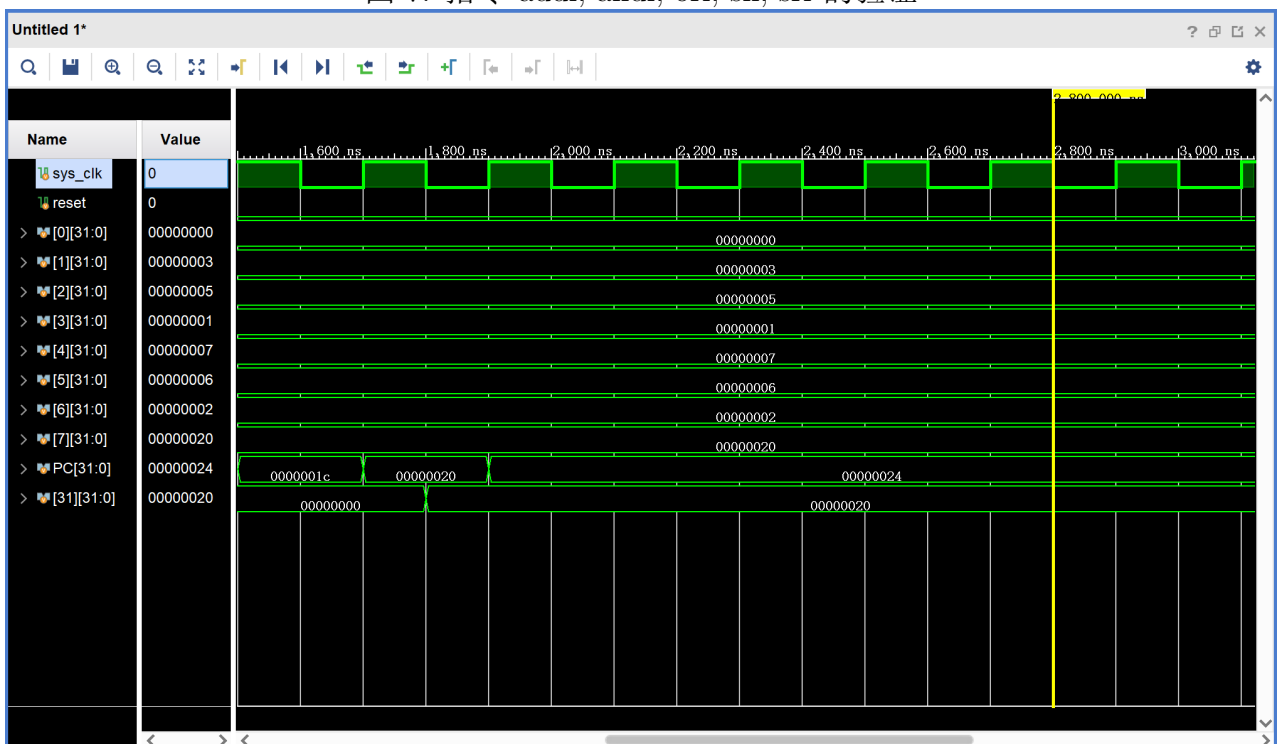


图 8: 指令 jr, jal 的验证

4.2 内存设置

4.2.1 数据内存设置

根据乘法器内容，将数据内存设为：第 1 个字为 9，第 2 个字为 d，表示 2 个乘数，剩余内容全部设为 0 即可。

```
E: > Desktop > Project5-Multiplier > Project5.sim > sim_1 > behav > xsim > ≡ lab05dat_tb
1      00000009
2      0000000d
3      00000000
4      00000000
5      00000000
6      00000000
7      00000000
8      00000000
```

图 9: 乘法器仿真的数据内存

4.2.2 指令内存设置

检查指令语义后发现，各指令对应的机器码与本次实现的处理器工作原理相符，可直接利用测试用例中的机器码初始化指令内存，其中每一行对应汇编代码中的一条指令，故不再赘述各指令含义：

```
E: > Desktop > Project5-Multiplier > Project5.sim > sim_1 > behav > xsim > ≡ lab05instr_tb
1      10001100000000001000000000000000
2      100011000000000010000000000000100
3      00000000000000001000100000001000010
4      00000000000000001000010100001000000
5      000100000010001010000000000000001
6      0000000010000000101000000000100000
7      00000000000000001000010000001000010
8      000000000000000010000100001000000
9      000100000010000000000000000000001
10     000010000000000000000000000000010
11     1010110000000100000000000000001000
```

图 10: 乘法器仿真的指令内存

4.3 仿真实现与结果展示

沿用前述仿真过程中编写的 `instr_16_tb.v` 作为激励文件即可，运行仿真，结果如图所示，在第 33 周期得到了乘法运算的结果 117：

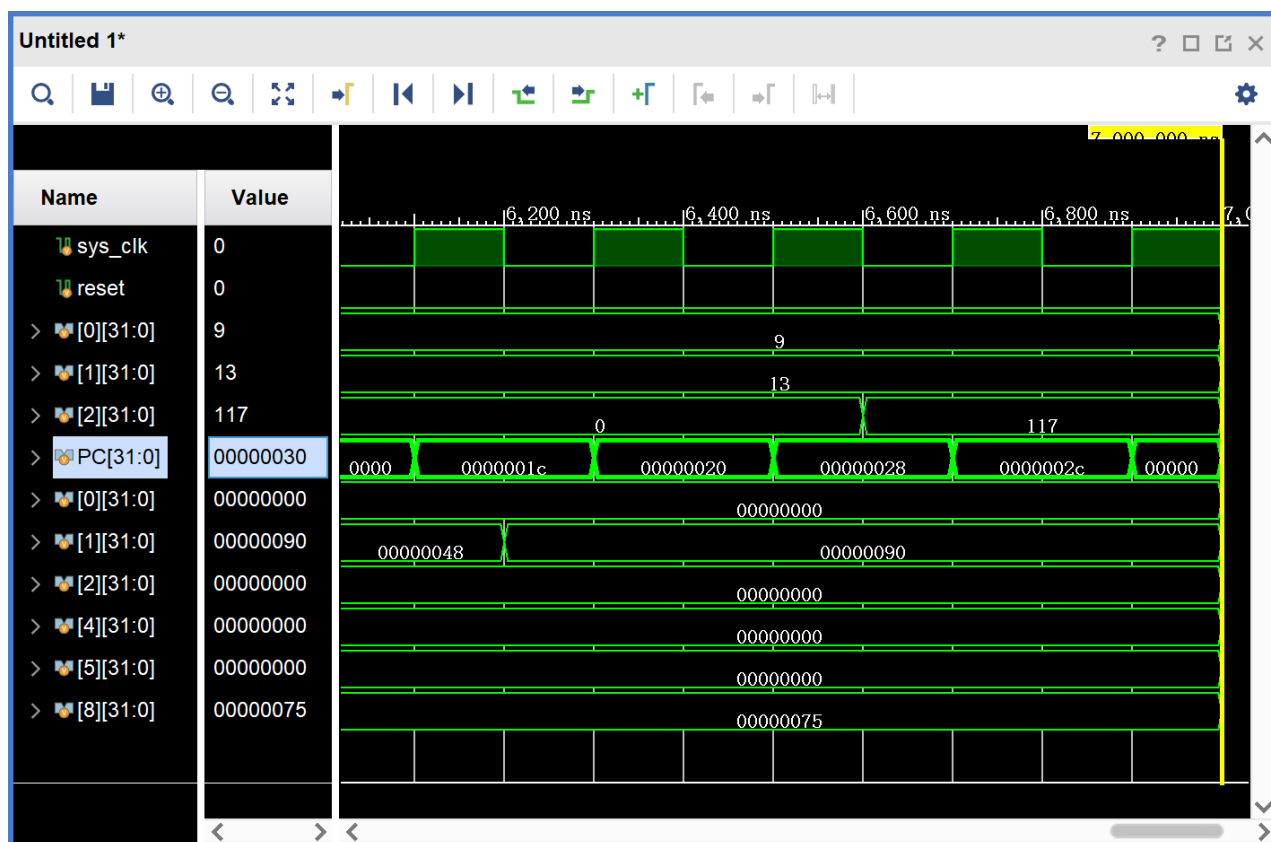


图 11: 乘法器仿真的结果展示

以上仿真表明本实验实现的 CPU 能够正确完成乘法器的功能。

5 上板验证

5.1 设计文件的修改

5.1.1 显示模块的添加及 CPU 的相应修改

在 Lab02 中找到 display 模块 (`display.v`, `display.edif`) 作为 IP 核，添加到 Lab05 的 Design Source 中。

分别找到 Lab01 中 `reset` 的管脚约束和 Lab02 中操作数读取、LED 灯、7 段数码管的管脚约束，添加至 Lab05 的 Constraints 中，如图12所示。

并根据 Lab01, Lab02 的实验内容和管脚约束文件的配置，对已有 CPU 模块进行添加和修改，具体修改内容如下。

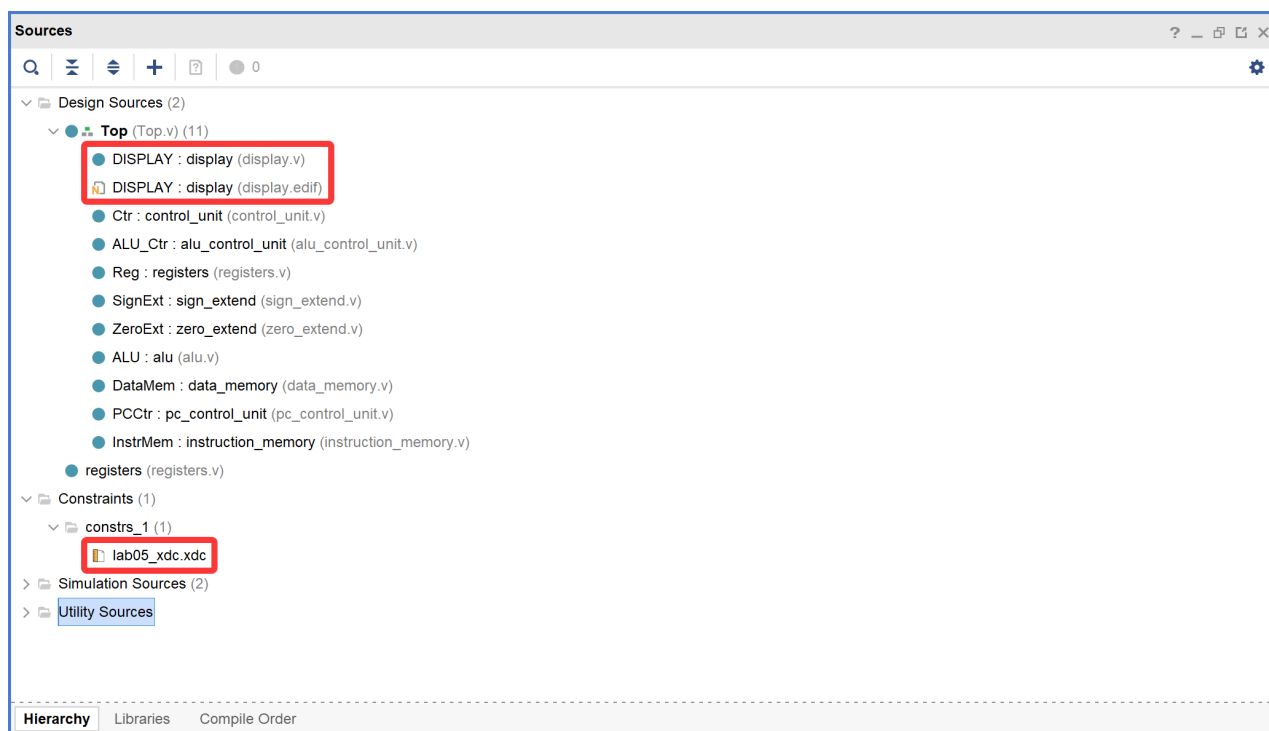


图 12: 上板验证的资源添加

Top 模块修改如下: (在 Top.v 中)

```
module Top(  
    input clk_p,  
    input clk_n,  
    input [3:0] op1,  
    input [3:0] op2,  
    input _reset,  
  
    output led_clk,  
    output led_do,  
    output led_en,  
  
    output wire seg_clk,  
    output wire seg_en,  
    output wire seg_do  
);  
  
wire Reset;  
assign Reset = !_reset;  
wire clk_i;
```

```
IBUFGDS IBUFGDS_inst (  
    .O(clk_i),  
    .I(clk_p),  
    .IB(clk_n)  
);  
reg [1:0] div;  
always @ (posedge clk_i) div <= div + 1;  
  
wire Clk;  
assign Clk = div[1];  
  
wire [3:0] Op1;  
wire [3:0] Op2;  
wire [7:0] Sum;
```

首先是对 reset 机制的修改：上板过程中 reset 拉起为 1，按下为 0，与模块实现中 reset 逻辑恰好相反，但由于本次实现的模块较多，因此在内部保留 Reset，只要对输入信号 _reset 取反送入 Reset 内部数据通路即可。

其次是差分时钟的设计与时钟分频：与 Lab02 中类似，频率为 50MHz 的 clk_i 在每个上升沿令 2 位 div 寄存器的值增加 1，并取 div 寄存器的高位作为内部时钟数据通路的数据源，则最终可在 Clk 得到频率为 25MHz 的系统时钟频率。

最后添加数据通路 Op1, Op2, Sum，用于展示操作数和运算结果。

display 模块设计如下：（在 Top.v 中）

```
display DISPLAY (  
    .clk(Clk),  
    .rst(1'b0),  
    .en(8'b01010011),  
    .data({4'b0, Op1, 4'b0, Op2, 8'b0, Sum}),  
    .dot(8'b0),  
  
    .led(~{Op1, Op2, Sum}),  
    .led_clk(led_clk),  
    .led_en(led_en),  
    .led_do(led_do),  
    .seg_clk(seg_clk),  
    .seg_en(seg_en),  
    .seg_do(seg_do)  
);
```

clk 表示时钟；en 表示 7 段数码管使能，写为 01010011 表示从左向右第 2, 4, 7, 8 个 7 段数码管亮起，其余数码管强制熄灭，即实现分隔亮起效果。

Op1, Op2, Sum 分别是 4, 4, 8 位 2 进制数，分别表示第 1 个操作数，第 2 个操作数，第 3 个操作数，用于将计算式显示到 7 段数码管和 LED 上。

data 表示 7 段数码管将显示的数据，data 每 4 位对应 1 个 7 段数码管，要对应上述分隔亮起的使能信号，依次将操作数 1 传到第 2 个 7 段数码管，将操作数 2 传到第 4 个 7 段数码管，运算结果传到第 7, 8 个 7 段数码管显示。dot 为 8 个 7 段数码管的小数点，本次实现的乘法器没有用到，直接设为 0。

led 表示 7 段数码管下方的 LED 灯，用于以 2 进制形式显示操作数和运算结果。

管脚约束文件调整如下：（在 lab05_xdc.xdc 中）

```
set_property PACKAGE_PIN AA12 [get_ports {op1[3]}]
set_property PACKAGE_PIN AA13 [get_ports {op1[2]}]
set_property PACKAGE_PIN AB10 [get_ports {op1[1]}]
set_property PACKAGE_PIN AA10 [get_ports {op1[0]}]
set_property IOSTANDARD LVCMOS15 [get_ports {op1[3]}]
set_property IOSTANDARD LVCMOS15 [get_ports {op1[2]}]
set_property IOSTANDARD LVCMOS15 [get_ports {op1[1]}]
set_property IOSTANDARD LVCMOS15 [get_ports {op1[0]}]

set_property PACKAGE_PIN AD10 [get_ports {op2[3]}]
set_property PACKAGE_PIN AD11 [get_ports {op2[2]}]
set_property PACKAGE_PIN Y12 [get_ports {op2[1]}]
set_property PACKAGE_PIN Y13 [get_ports {op2[0]}]
set_property IOSTANDARD LVCMOS15 [get_ports {op2[3]}]
set_property IOSTANDARD LVCMOS15 [get_ports {op2[2]}]
set_property IOSTANDARD LVCMOS15 [get_ports {op2[1]}]
set_property IOSTANDARD LVCMOS15 [get_ports {op2[0]}]
```

因为 Top 模块修改了输入操作数的命名，所以管脚约束中的读数据的目标位置也要修改为 Top 模块的 2 个 4 位输入 op1 和 op2；原理即为拨动 16 位开关表示操作数，内部模块读取 16 位开关状态，作为 2 个操作数输入的管脚约束。

5.1.2 内存的配置

因为上板过程中不能从文件中读取指令内存和数据内存的值，因此无需编写读取内存内容的源文件。将指令内存的内容在 Instruction Memory 模块的 initial 语句块中直接指定（相比于乘法器仿真中的汇编代码，额外添加了 1 条无条件跳转到本身的 j 指令，防止 PC 无限增加出现问题）：

```

initial begin
    mem_cont[0] = 32'b10001100000000001000000000000000;
    mem_cont[1] = 32'b1000110000000000100000000000000100;
    mem_cont[2] = 32'b0000000000000000100010000001000010;
    mem_cont[3] = 32'b00000000000000001000010100001000000;
    mem_cont[4] = 32'b0001000001000101000000000000000001;
    mem_cont[5] = 32'b0000000010000000101000000000100000;
    mem_cont[6] = 32'b0000000000000000100001000001000010;
    mem_cont[7] = 32'b000000000000000010000100001000000;
    mem_cont[8] = 32'b0001000001000000000000000000000001;
    mem_cont[9] = 32'b0000100000000000000000000000000010;
    mem_cont[10] = 32'b10101100000010000000000000000001000;
    mem_cont[11] = 32'b00001000000000000000000000000001011;
end

```

而对于数据内存，前 2 个字直接从 16 位开关中读取作为操作数，第 3 个字为运算结果，并将前 3 个字的内容送显示：

```

// Data Memory
assign display_1 = mem_cont[0][3:0];
assign display_2 = mem_cont[1][3:0];
assign result = mem_cont[2][7:0];

always @ (negedge clock) begin
    if (reset) begin
        mem_cont[0] <= operand_1;
        mem_cont[1] <= operand_2;
        mem_cont[2] <= 0;
    end
    else if (mem_wr) mem_cont[addr>>2] = wr_data;
end

```

其中 display_1, display_2, result 为内存送入 display 模块显示的数据，operand_1, operand_2 为数据内存初始化的数据源，与 16 位开关中的 8 位相连。除此之外，要保留 CPU 中数据内存写入功能，因为乘法运算最后 1 步要将结果写回内存。

此外还要调整 Top 模块中数据内存相关的数据通路的连接：

```

// Top
wire [31:0] RdDataMem;
data_memory DataMem(
    .reset(Reset),

```



```
.clock(Clk),  
.addr(ALUResult),  
.mem_wr(MemWrite),  
.wr_data(RegData2),  
.mem_rd(MemRead),  
.operand_1(op1),  
.operand_2(op2),  
.display_1(Op1),  
.display_2(Op2),  
.result(Sum),  
.rd_data(RdDataMem)  
);
```

5.1.3 最后调整

在生成二进制流文件过程中产生了报错信息，经查阅资料得知，错误产生为多驱动所致，如 Top 模块中，时钟下降沿更新 PC 的逻辑的修改前后对比如下，其中 PC 重置操作在指令实现过程中已由 PC Control Unit 外置到 Top 模块。

修改前，PC 更新对 Reset 和 Clk 上升沿均敏感，产生了多驱动错误：

```
always @ (Reset) PC<=0;  
always @ (posedge Clk) PC <= NewPC;
```

修改后，PC 只对 Clk 上升沿敏感，PC 正常更新和重置操作均在 PC 上升沿完成，避免多驱动错误：

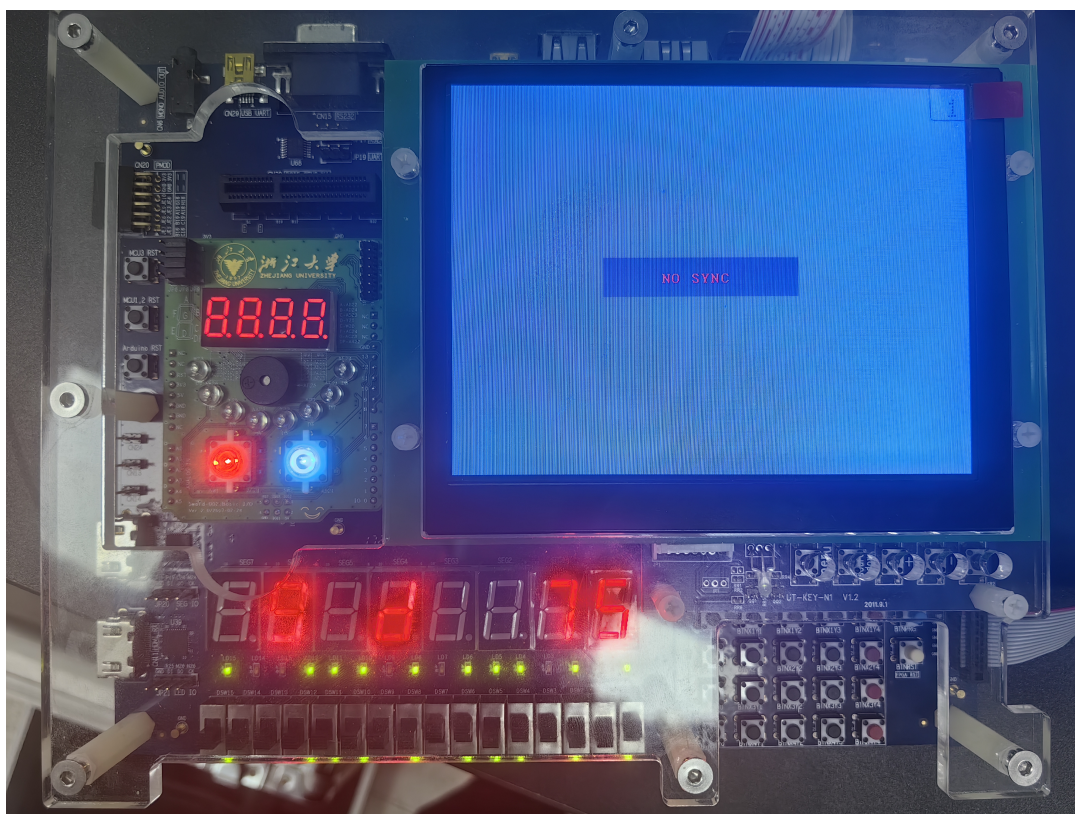
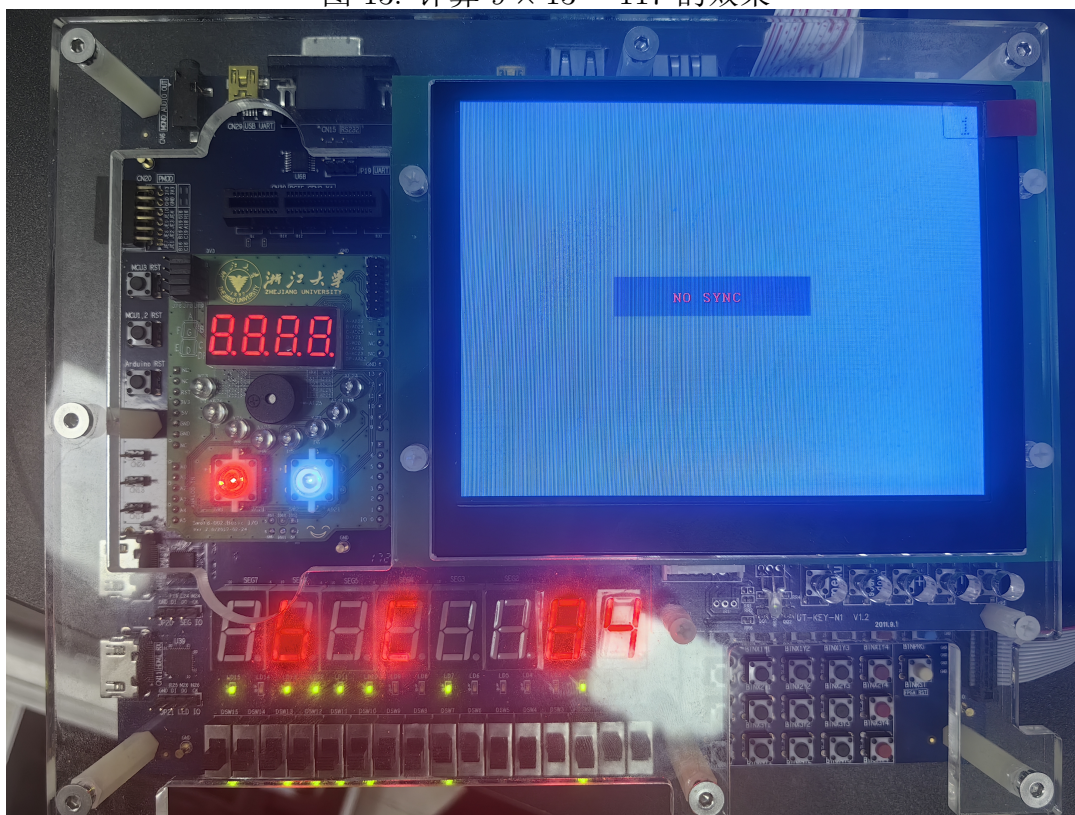
```
always @ (posedge Clk) begin  
    if (Reset) PC <= 0;  
    else PC <= NewPC;  
end
```

此外还有多处多驱动错误修正，在这里不再一一赘述。

5.2 结果展示

拨码计算 $9H \times dH = 75H \Leftrightarrow 9 \times 13 = 117$ 的结果如图13所示，拨码计算 $bH \times cH = 84H \Leftrightarrow 11 \times 12 = 132$ 的结果如图14所示，可以看到 2 次拨码乘法中，7 段数码管显示的 16 进制计算式和 LED 表示的 2 进制计算式都能显示预期的效果。

以上现象表明 Lab05 工程成功实现了拨码乘法器。

图 13: 计算 $9 \times 13 = 117$ 的效果图 14: 计算 $11 \times 12 = 132$ 的效果

6 总结与反思

本次实验我由简单到复杂，从实现并验证支持 9 条指令的处理器开始，实现并验证了支持 16 条指令的单周期处理器。最后以乘法器汇编代码为例，仿真实现和上板验证了支持 16 条指令单周期处理器的功能。在完成本实验的过程中，我对计算机系统结构，尤其是 CPU 的实现方式有了更深刻的认识，对硬件开发流程的了解也愈加深入。

本实验期间为了扩展功能，我对各个部件和控制信号进行了修改，并添加了部分控制信号以支持更复杂的功能。这要求我不能对 Verilog 代码一知半解，而是要对每一条语句的功能有清晰的认识，否则就会像本实验一样，过程中出现调试效率低、对 bug 产生的原因并不明确等问题。比如因为不了解 `=` (非阻塞赋值) 和 `<=` (阻塞赋值) 的差异，导致首次仿真波形中出现了大量的 Z (高阻态)，并在修复此问题上花费了大量时间。

从本实验来看，前述 Lab03 和 Lab04 中部分模块的设计在逻辑上仍有许多不足之处，这启示我在以后的子模块设计中，每时每刻都要考虑到子模块在整个系统中的作用，避免诸如多驱动错误导致生成二进制流文件时报错、时序逻辑问题导致仿真波形出现高阻抗或无关位等情况的发生。这些问题需要在接下来的 Lab06 中加以改正。